IMPACTANALYSE

US-144 — Catalogusoverzicht filteren op alleen voorradige artikelen

Inhoudsopgave

1. Bevindingen 2
2. Scenario 3
3. Bijlage 1: Technische uitwerking 5

Bevindingen

Huidige situatie

De Catalog.API-service van de eShop biedt via `GET /api/catalog/items` (API-versie 2.0) een gepagineerd catalogusoverzicht, afgenomen door onder andere de Blazor-webapplicatie en de mobiele client. Het endpoint ondersteunt vandaag filteren op naam (`name`), typen (`type`) en merken (`brand`) en sorteert de resultaten vast op naam. Er is geen mogelijkheid om de resultaatset te beperken tot artikelen met beschikbare voorraad: elk catalogusartikel wordt teruggegeven, ongeacht de waarde van `AvailableStock`. Daardoor kunnen clients artikelen tonen die bij het afrekenen alsnog niet leverbaar blijken.

De paginering-metadata (`count`, `pageIndex`, `pageSize`) wordt berekend over dezelfde (ongefilterde)-query als de pagina met artikelen; die architectuur leent zich er direct voor om een extra filter op dezelfde query toe te passen.

Gewenste situatie:

`GET /api/catalog/items` (v2) accepteert een extra, optionele booleaanse query-parameter `inStock`. Wanneer `inStock=true` wordt meegegeven, bevat de respons uitsluitend artikelen met `AvailableStock` groter dan nul, waarbij de `count`-/paginering-metadata de gefilterde resultaatset weerspiegelt. Wordt de parameter weggelaten (of is deze `false`), dan retourneert het endpoint exact dezelfde artikelen, in dezelfde volgorde en met dezelfde paginering-metadata als vandaag; bestaande consumenten (Blazor-webapplicatie, mobiele client) merken niets van de wijziging en worden in deze story niet aangepast. Het filteren gebeurt server-side in de datanquery (EF Core), vóór de tellings- en paginastappen, en is combineerbaar met de bestaande filters voor naam, type en merk. Een ongeldige waarde voor de parameter leidt tot een HTTP 400 (Bad Request) in plaats van een 500.

Scenario

Drie opties zijn afgewogen:

1. **Extra optionele query-parameter op het bestaande overzicht-endpoint (gekozen).** De filterlogiek sluit naadloos aan op de bestaande queryopbouw in `GetAllItems`: elke filter wordt als `Where`-clause op dezelfde `IQueryable<CatalogItem>` toegepast vóórdat `LongCountAsync()` en `Skip`/`Take` worden uitgevoerd. Hierdoor is AC "paginering-metadata volgt de gefilterde set" vrijwel gratis, en is de wijziging per definitie backwards-compatibel (default: geen filter). Geringste risico, minimale code, geen nieuw oppervlak.
2. **Afzonderlijk endpoint (bijvoorbeeld `GET /api/catalog/items/available`).** Verwerpelijk: duplicatie van de paginerings-, sorteer- en filterlogiek of een fragiele interne delegatie, een nieuw API-oppervlak om te documenteren en te onderhouden, en het gecombineerd gebruik met `name`/`type`/`brand` wordt onnodig omslachtig.
3. **Berekende "is available"-kolom of aanpassing van het `CatalogItem`-datamodel.** Uitdrukkelijk buiten scope van de story (geen databasemigratie, geen datamodelwijziging). Daarnaast overbodig: de beschikbare voorraad (`AvailableStock`) is al een bestaande, bevraagbare kolom.

Een variant waarin ná het pagineren wordt gefilterd, is geen serieus scenario: die levert lege of te korte pagina's op en miskent de acceptatie-eis dat de filtering in de datanquery zelf plaatsvindt.

Randvoorwaarden en uitgangspunten

- Deze impactanalyse heeft een geldigheid van 2 maanden na het versturen van de definitieve versie en zal herzien moeten worden op het moment dat er tussentijds andere wijzigingen op het informatiesysteem uitgevoerd worden.
- De nieuwe functionaliteit wordt uitsluitend toegevoegd aan API-versie 2.0 van het overzicht-endpoint. API-versie 1.0 (`GetAllItemsV1`) blijft ongewijzigd en degradeert naar het huidige gedrag; bestaande consumenten die v1 gebruiken, ondervinden geen enkele verandering.
- Geen wijzigingen aan de Blazor-webapplicatie, de mobiele client of enige andere consument; geen wijziging aan het `CatalogItem`-datamodel en geen databasemigratie.
- Beschikbaarheid betekent in deze story: `AvailableStock > 0`. De velden `OnReorder` en `RestockThreshold` spelen geen rol.
- **Aanname base branch (ter bevestiging bij plan-review):** dit story-item draagt geen `feature:{naam}`-tag en er bestaat geen `docs/requirements/`-feature-map voor deze story-serie (US-01 t/m US-03). De architect heeft daarom de featurenaam `catalog-in-stock-filter` vastgesteld en de releasebranch `release/catalog-in-stock-filter` (op `main`) lazy aangemaakt, waarop de storebranch `feature/US-144-catalog-items-in-stock-filter` is gebaseerd. Moet een mens bij plan-review een andere featurenaam hanteren, dan worden deze branches tegen die licht herbenoemd; de technische uitwerking verandert daar niet door.
- Oplevering en goedkeuring: de implementatie wordt op de storebranch opgeleverd met een PR naar de releasebranch; Quality Assurance beoordeelt code en acceptance-criteria, waarna een mens de PR na `human-verify` squash-mergt. De OpenAPI-beschrijving `Catalog.API_v2.json` wordt als gegenereerd artefact meegecommit.
- Testen: de functionele tests in `tests/Catalog.FunctionalTests` (MSTest, bestaand patroon met Aspire-Postgres-fixture) zijn de acceptatietest-laag van dit informatiesysteem; er bestaan geen Gherkin/Reqnroll-specs in deze repository. Alle tests moeten slagen zowel lokaal als in de CI-pijplijn voordat de oplevering als gereed geldt.

Security

- Het betreft een uitsluitend-uitleesbaar, publiek catalogusendpoint binnen de eShop-demo-architectuur; er verandert niets aan authenticatie, autorisatie of CORS-configuratie.
- De nieuwe query-parameter is een strikt getypeerde `bool` die door de minimal-API-modelbinding via `TryParse` wordt gevalideerd; een ongeldige waarde leidt tot een 400 Bad Request zonder interne details (conform bestaand foutgedrag).
- De filtering gebeurt via LINQ/EF Core en wordt daarmee genormaliseerd en geparameteriseerd naar SQL omgezet; er ontstaat geen injectie-aanvalsvlak (OWASP A03 Injection, GASD14).
- Er worden geen nieuwe databronnen, secrets, externe oproepen of persoonsgegevens geïntroduceerd; de BIV-classificatie van het informatiesysteem blijft onveranderd.

Risico's

Risico	Impact	Kans van optreden	Maatregel
Bestaande consumenten (WebApp, mobiele client) vertonen onbedoeld gewijzigd gedrag door de nieuwe code	Middel	Laag	Parameter is optioneel met default `false` (geen filter); regressietesten die het niet-opgegeven/`false`-gedrag vergelijken met de huidige output; bestaande `CatalogApiTests` blijven ongewijzigd slagen
Clients roepen `inStock` per ongeluk af op API-versie 1.0 en krijgen een ongefilterde lijst	Laag	Laag	Feature is opzettelijk v2-only; expliciet vastgelegd in randvoorwaarden en in de OpenAPI-beschrijving van v2
Testdata-pollution: de nieuwe functionele tests aanmaken/verwijderen van artikelen in de gedeelde test-database beïnvloedt andere tests	Laag	Middel	Unieke test-ids buiten alle bestaande id-bereiken, filters op unieke naam-prefix en opschonen (DELETE) in `finally`, volgens het bestaande patroon in `CatalogApiTests`
Prestatie-impact van de extra WHERE-clause op grote catalogi	Laag	Laag	Bestaande kolom, geen nieuwe index of migratie nodig voor de omvang van deze catalogus; bij groei later alsnog index-meeting (niet noodzakelijk voor deze story)

Beschikbaarheid

Naar verwachting geen invloed op de beschikbaarheid: het is een uitbreidende (additive) API-wijziging zonder databasemigratie, zonder configuratiewijzigingen en zonder downtime bij uitrol. Oude en nieuwe versie van de service zijn onderling uitwisselbaar; roll-back is een eenvoudige herstart op de vorige image.

Bewaartermijn informatie

Niet van toepassing. De wijziging is uitsluitend-uitleesbaar: er wordt geen data toegevoegd, gemuteerd of verwijderd, dus er is geen effect op opslag- of verwijderafspraken.

Impact

De technisch inhoudelijke impact is vastgesteld op **8 uur (indicatie, door architect bepaald — behoeft validatie door reviewer)**:

- Maken impactanalyse
- Maken implementatie-/test-/rollbackplan
- Review impactanalyse en implementatie-/test-/rollbackplan
- Implementatie (acceptatie/productie)
- Testplan uitvoeren
- Rollback uitvoeren
- Communicatie
- Documentatie en/of CMDB bijwerken

De rollback is een herstart/terugdraaien naar de vorige containerimage van de Catalog.API-service; er zijn geen databasemutaties of configuratiewijzigingen die apart teruggezet moeten worden.

Bijlage 1: Technische uitwerking

Betrokken componenten

- `src/Catalog.API/Apis/CatalogApi.cs` — het overzicht-endpoint `GetAllItems` (API-v2) en de afgeleide V1-endpoints die erop delegeren.
- `src/Catalog.API/Catalog.API_v2.json` — gegenereerde OpenAPI-beschrijving (build-time artefact, wordt meegecommit).
- `tests/Catalog.FunctionalTests/CatalogApiTests.cs` — functionele acceptatietesten.
- **Niet** betrokken: `CatalogItem`-datamodel, `CatalogContext`, migraties, WebApp/ClientApp/HybridApp, `GetCatalogFacets` (facet-tellingen blijven in deze story ongefilterd; buiten scope).

Backend implementatiestappen

1. Voeg in `CatalogApi.GetAllItems` (de handler achter `v1.MapGet("/items", ...)` voor v2, regel ~131) als laatste parameter toe: `[Description("When true, only return items that are in stock (available stock greater than zero)")] bool inStock = false`. Door de defaultwaarde blijft de signatuur backwards-compatibel voor de bestaande interne aanroepen (`GetAllItemsV1`, `GetItemsByName`, `GetItemsByBrandAndTypeId`, `GetItemsByBrandId`), die de parameter niet doorgeven en dus `false` krijgen.

2. Pas de filter toe op de query-root, direct na de bestaande `brand`-filter en vóór `LongCountAsync()`:

   ```
   if (inStock)
   {
       root = root.Where(c => c.AvailableStock > 0);
   }
   ```

   Doordat zowel `totalItems` (de `LongCountAsync`) als `itemsOnPage` (de `Skip`/`Take`-pagina) van dezelfde `root` afstammen, is het filter automatisch server-side en correct meegenomen in de paginering-metadata (AC's over total-count en combineerbaarheid).

3. Laat `GetAllItemsV1` en de overige delegerende handlers ongewijzigd (ze krijgen impliciet `inStock: false`).

4. Valideer het 400-gedrag: de minimal-API-modelbinding bindt `bool` via `TryParse`; een waarde als `inStock=abc` moet zonder extra code een 400 opleveren. Bevestig dit met de functionele test hieronder (acceptatiecriterium); mocht de binding onverhoopt toch deaulten, valideer dan expliciet op de endpoint.

5. Bouw `src/Catalog.API` en commit het opnieuw gegenereerde `Catalog.API_v2.json` (nieuwe optionele `inStock`-parameter onder `GET /api/catalog/items`, versie 2.0). `Catalog.API.json` (v1) hoort ongewijzigd te blijven — verifieer dat in de diff.

Frontend implementatiestappen

Niet van toepassing. Geen enkele UI roept de nieuwe parameter af; de WebApp en de mobiele client blijven exact zoals vandaag aanroepen (uitdrukkelijk buiten scope van de story).

API contracts

`GET /api/catalog/items?pageSize={n}&pageIndex={i}&name={s}&type={t}&brand={b}&inStock={bool}` (API-versie 2.0)

Parameter	Type	Verplicht	Default	Gedrag
`inStock`	`boolean`	Nee	`false`	`true`: alleen artikelen met `AvailableStock > 0`. `false`/weggelaten: huidig gedrag. Ongeldige waarde: `400 Bad Request` (`application/problem+json`). Combineerbaar met `name`, `type`, `brand` en de vast-op-naam-sortering.

Responsstructuur (`PaginatedItems<CatalogItem>`) is ongewijzigd; `count` is bij `inStock=true` het aantal artikelen íén de gefilterde set. De overige catalogus-endpoints (`/items/{id}`, `/items/by`, `/items/facets`, semantic relevance, v1-overzicht) zijn ongewijzigd.

Database wijzigingen

Geen. Het filter gebruikt de bestaande kolom `availablestock` op tabel `catalog` (`CatalogItems`); geen entities, geen migraties, geen nieuwe index voor de huidige catalogusomvang.

Testverwachtingen

Functionele tests in `tests/Catalog.FunctionalTests/CatalogApiTests.cs`, naar het bestaande MSTest-patroon (`[Theory]` op API-versie waar zinvol; nieuwe tests gericht op v2.0). De standaard-seed bevat uitsluitend artikelen met `AvailableStock = 100`; de tests voor `inStock=true` moeten zelf een mix creëren via `POST /api/catalog/items` (met unieke id's buiten de bestaande testbereiken 10015/10016 en 99901/99902, unieke naam-prefix, en DELETE-opschonen in `finally`):

1. **Parameter weggelaten / `inStock=false` is byte-op-byte huidig gedrag**: zelfde pagina id's, zelfde volgorde, zelfde `count` als een aanroep zonder de parameter (beschermt AC "geen gedragsverandering"; bestaande tests blijven bovendien ongewijzigd slagen).
2. **`inStock=true` filtert out-of-stock**: creëer o.a. één artikel met `AvailableStock = 0` en één met voorraad onder een unieke naam-prefix; verifieer dat `count` en elke `data`-item van `?name={prefix}&inStock=true` uitsluitend artikelen met `AvailableStock > 0` bevatten en het 0-voorraadartikel ontbreken.
3. **Gecombineerd met paginering**: creëer bijvoorbeeld 3 voorradige en 2 niet-voorradige artikelen met dezelfde prefix; vraag met `pageSize=2` pagina's op en verifieer dat `count` = 3 is, pagina 0 twee items bevat en pagina 1 één (nooit een lege/te korte pagina midden in de gefilterde lijst).
4. **Combineerbaar met bestaande filters**: `inStock=true` samen met `type`/`brand`/`name` geeft de doorsnede van alle filters terug.
5. **Ongeldige parameterwaarde**: `?inStock=abc` levert `400 Bad Request` (geen 500).

Geen unit tests nodig bovenop deze functionele tests (de wijziging is één LINQ-clause binnen een endpoint-handler; de functionele laag dekt het volledige gedrag). De bestaande tests in `Catalog.FunctionalTests` en de overige testprojecten moeten ongewijzigd slagen (regressie-eis).